

Σ PREFIX SUM

DSA Pattern — Episode 26

Concept → Flowchart → Template → Problem. Everything from the video + extra insights.

Covers: What is Prefix Sum • Suffix Sum • Flowchart (4 Categories) • Sliding Window Limitation • Pivot Index (LC 724) • Template

Table of Contents

1. What is Prefix Sum? (Concept, Not Algorithm)
2. The Flowchart — 4 Categories of Questions
3. Why Sliding Window Fails on Negatives
4. Prefix Sum Formula & Building the Array
5. Suffix Sum — The Mirror Concept
6. The Key Insight: $\text{Left} + \text{arr}[i] + \text{Right} = \text{Total Sum}$
7. Problem: Find Pivot Index (LC 724)
8. Optimized Approach — No Extra Arrays
9. Complete C++ Code + Dry Run
10. The 4-Step Template (Works for ALL prefix problems)

1. What is Prefix Sum?

The Core Concept

Prefix means '**before**'. Sum means Sum. So Prefix Sum = '**sum of everything before index i**'.

At any index **i**, the prefix sum is: $\text{arr}[0] + \text{arr}[1] + \dots + \text{arr}[i-1]$

This is NOT an algorithm. It is a **CONCEPT** / **PATTERN** that forms the basis of many questions.

Example → $\text{arr} = [1, 2, 3, 4, 5]$ (indices 0–4)

Index i	prefix[i] (sum of everything before i)
prefix[0]	0 (nothing before index 0)
prefix[1]	arr[0] = 1

prefix[2]	$\text{arr}[0] + \text{arr}[1] = 1 + 2 = 3$
prefix[3]	$\text{arr}[0] + \text{arr}[1] + \text{arr}[2] = 1 + 2 + 3 = 6$
prefix[4]	$\text{arr}[0] + \text{arr}[1] + \text{arr}[2] + \text{arr}[3] = 1 + 2 + 3 + 4 = 10$

💡 Why 'before' and not 'including'?

prefix[i] stores the sum of elements BEFORE index i. This means the current element **arr[i]** can use prefix[i] to make a decision about its own position — it knows everything that came before, without counting itself.

🚫 prefix[0] is always 0 — Why?

There is nothing before index 0. So $\text{prefix}[0] = 0$ (empty sum). This is the base case for the entire pattern.

📦 Prefix Sum works ONLY on Arrays

This entire pattern applies exclusively to arrays — not strings, not linked lists, not trees. The concept requires numeric elements that can be summed.

2. The Flowchart — 4 Categories of Questions

Every prefix sum question fits into exactly ONE of these 4 buckets. Nothing outside this flowchart. Instructor's guarantee.

✅ Is it an Array + Subarray question?

This MUST be true first. If the question is about arrays and subarrays, proceed. Otherwise prefix sum doesn't apply.

Category (What is asked?)	What you need
🟢 Category 1: Left/Right (Pivot/Equilibrium) Find index where left sum == right sum (or some condition between them) Difficulty: EASY	Data Structure Needed: Simple variable (left) or simple array No hashmap needed
🟡 Category 2: Exact Sum == k OR Sum % k == 0 Count/find subarrays with exact sum k, or sum divisible by k Negatives allowed Difficulty: MEDIUM	Data Structure Needed: HashMap (prefix sum → frequency) Core of classic prefix sum pattern
🟠 Category 3: Shortest Window ≥ k (with negatives) Shortest subarray with sum ≥ k, array	Data Structure Needed: Deque (monotonic deque on prefix array) See note below

has negatives (Positive-only? → Sliding Window) Difficulty: MEDIUM-HARD	
 Category 4: Range Sum Queries Find count of subarrays where sum falls in a range [lo, hi] Difficulty: HARD (Google-level)	Data Structure Needed: Merge Sort on Prefix Array (Covered separately)

 Negatives are the KEY differentiator

If the array has only positive numbers: → Sliding Window works for most subarray sum problems

If the array has negative numbers: → Sliding Window breaks. Use Prefix Sum (HashMap or Deque depending on category)

3. Why Sliding Window Fails on Negatives

Sliding Window works on the assumption that adding elements increases the sum and removing elements decreases it. Negatives destroy this assumption.

Sliding Window's Decision Logic

Action	What Sliding Window does
Want to INCREASE sum	Expand `high` pointer (add elements to the right)
Want to DECREASE sum	Shrink `low` pointer (remove elements from the left)

 What breaks with negatives?

Scenario 1: `low` points to a negative element. Shrinking `low` (removing it) INCREASES the sum instead of decreasing it. Decision logic inverted.

Scenario 2: Expanding `high` to include a negative element DECREASES the sum instead of increasing it. Decision logic inverted again.

Result: The entire shrink/expand strategy is unpredictable. Sliding Window completely fails.

 Real Interview Tip

If you see a subarray question and spot negative numbers, immediately think: "**Sliding Window won't work here. I need Prefix Sum.**"

Also: notice that every Sliding Window question you've done probably had only positive numbers. That's not an accident.

4. Prefix Sum Formula & Building the Array

Naive Approach — Two nested loops (works but slow)

For each index i , loop from 0 to $i-1$ and sum everything. This is $O(N^2)$.

Efficient Pattern — Use the previous prefix value

Notice the recurrence: each prefix sum is just the **previous prefix sum + previous array element**

```
prefix[0] = 0                                (base case)

prefix[i] = prefix[i-1] + arr[i-1]           (for i >= 1)

i.e. prefix[i] = prefix[i-1] + arr[i-1]
```

Why arr[i-1] and not arr[i]?

prefix[i] stores the sum of everything **BEFORE** index i . So at index i , the most recent element that's 'before' is $arr[i-1]$. That's what we add.

C++ Code — Building the Prefix Array

```
vector<int> buildPrefixArray(vector<int>& arr) {
    int n = arr.size();
    vector<int> prefix(n, 0); // Initialize all to 0
    // prefix[0] = 0 already (base case)

    for (int i = 1; i < n; i++) {
        prefix[i] = prefix[i-1] + arr[i-1];
        //           ↑ previous prefix   ↑ arr element before i
    }
    return prefix;
}

// Example: arr = [1, 2, 3, 4, 5]
// prefix = [0, 1, 3, 6, 10]
// prefix[0]=0 prefix[1]=1 prefix[2]=3 prefix[3]=6 prefix[4]=10
```

What prefix[i] represents

prefix[i] = sum of all elements from index **0** to **i-1** (inclusive)
Or equivalently: the sum of the left side of index i, not counting i itself.

5. Suffix Sum — The Mirror Concept

Suffix Sum is exactly Prefix Sum, but from the other direction. Same concept, same logic, just reversed.

Definition

suffix[i] = sum of all elements from index **i+1** to **n-1** (everything AFTER index i)
suffix[n-1] = 0 (nothing after the last element)

suffix[n-1] = 0 (base case)

suffix[i] = suffix[i+1] + arr[i+1] (for i <= n-2)

← Exact mirror of prefix formula, going right to left

CRITICAL: Build Suffix Array from RIGHT to LEFT

Prefix array: Build left to right (i = 0, 1, 2, ... n-1). Each entry needs the previous one.

Suffix array: Build right to left (i = n-2, n-3, ... 0). Each entry needs the next one.

If you build suffix left to right, you'll use values you haven't computed yet — it will crash or give wrong results!

```
vector<int> buildSuffixArray(vector<int>& arr) {
    int n = arr.size();
    vector<int> suffix(n, 0); // Initialize all to 0
    // suffix[n-1] = 0 already (base case)

    for (int i = n-2; i >= 0; i--) { // ← Right to left!
        suffix[i] = suffix[i+1] + arr[i+1];
        //           ↑ next suffix   ↑ arr element after i
    }
    return suffix;
}

// Example: arr = [1, 2, 3, 4, 5]
```

```
// suffix = [14, 12, 9, 5, 0]
// suffix[0]=14 suffix[1]=12 suffix[2]=9 suffix[3]=5 suffix[4]=0
```

Name	Meaning	Direction	Loop Order
prefix[i]	Sum of arr[0..i-1] (everything BEFORE i)	→ LEFT side	Built LEFT to RIGHT
suffix[i]	Sum of arr[i+1..n-1] (everything AFTER i)	→ RIGHT side	Built RIGHT to LEFT

6. The Key Insight: Left + arr[i] + Right = Total Sum

This single insight unlocks the optimized solution for ALL Category 1 problems.

Visual Proof

At any index **i**:

prefix[i] = arr[0] + arr[1] + ... + arr[i-1] ← everything LEFT of i

arr[i] = arr[i] ← the current element

suffix[i] = arr[i+1] + arr[i+2] + ... + arr[n-1] ← everything RIGHT of i

prefix[i] + arr[i] + suffix[i] = total sum of entire array

Verification — arr = [1, 7, 3, 6, 5, 6] (sum = 28)

Index	Left (prefix)	arr[i]	Right (suffix)	Check
i=0	prefix=0	arr=1	suffix=27	0 + 1 + 27 = 28 ✓
i=1	prefix=1	arr=7	suffix=20	1 + 7 + 20 = 28 ✓
i=2	prefix=8	arr=3	suffix=17	8 + 3 + 17 = 28 ✓
i=3	prefix=11	arr=6	suffix=11	11 + 6 + 11 = 28 ✓
i=4	prefix=17	arr=5	suffix=6	17 + 5 + 6 = 28 ✓
i=5	prefix=22	arr=6	suffix=0	22 + 6 + 0 = 28 ✓

Total = prefix[i] + arr[i] + suffix[i]

Therefore: $\text{suffix}[i] = \text{Total} - \text{prefix}[i] - \text{arr}[i]$

$\text{right} = \text{sum} - \text{left} - \text{arr}[i]$

← You can compute **RIGHT** on the fly if you already have **LEFT** and **Total**!



The Optimization This Unlocks

You do NOT need to build a separate suffix array. Compute `sum` once, then **compute right = sum - left - arr[i] on the fly** at every index. This eliminates one entire array!

7. Problem: Find Pivot Index (LC 724)



Problem Statement

LeetCode 724: Given an array of integers, find the leftmost pivot index.

A **pivot index** is any index where: $\text{prefix}[i] == \text{suffix}[i]$

i.e., the sum of all elements to the LEFT equals the sum of all elements to the RIGHT.

Return: the pivot index, or -1 if none exists.

Example: $\text{arr} = [1, 7, 3, 6, 5, 6] \rightarrow \text{Answer: } 3$

At index 3: Left sum = $1+7+3 = 11$, Right sum = $5+6 = 11$ ✓

Flowchart Check — Which Category?

- Array + Subarray? Yes ✓
- Left/Right sum condition? Yes — find where left sum == right sum ✓
- Category: **Category 1 — Simple variable/array**
- Negatives? Doesn't matter for this approach ✓

Approach 1 — Build Two Arrays (works, not optimal)

Build `prefix[]` and `suffix[]` arrays separately, then compare at each index.

```
// Approach 1: Two separate arrays – O(N) time, O(N) space
int pivotIndex_TwoArrays(vector<int>& nums) {
    int n = nums.size();
    vector<int> prefix(n, 0), suffix(n, 0);

    for (int i = 1; i < n; i++)
        prefix[i] = prefix[i-1] + nums[i-1];

    for (int i = n-2; i >= 0; i--) // Right to left!
        suffix[i] = suffix[i+1] + nums[i+1];

    for (int i = 0; i < n; i++)
        if (prefix[i] == suffix[i]) return i;

    return -1;
}
```

⚠ Why This Isn't the Best Approach

You're using **O(N) extra space** for two arrays. Many teachers stop here, but the interviewer WILL ask for space optimization. The right approach uses O(1) space.

8. Optimized Approach — No Extra Arrays

Use the insight: ...

No suffix array needed. Just a running **left** variable, updated as we loop.

How `left` is Updated at Each Step

i	left (prefix)	right (computed)	Action
i=0	left=0	right=28-1-0=27	0≠27, skip. Then left += nums[0] → left=1
i=1	left=1	right=28-7-1=20	1≠20, skip. Then left += nums[1] → left=8
i=2	left=8	right=28-3-8=17	8≠17, skip. Then left += nums[2] → left=11
i=3	left=11	right=28-6-11=11	11==11 ✅ return 3

⚡ Key Order of Operations

Step 1: Update **left** first (add arr[i-1] to it)

Step 2: Compute **right = sum - arr[i] - left**

Step 3: Check if left == right → return i

You add `arr[i-1]` to left because at position `i`, everything before it (up to index `i-1`) should be in the left sum.

9. Complete C++ Code + Dry Run

```
// LeetCode 724: Find Pivot Index
// Time: O(N) | Space: O(1) ← No extra arrays!

class Solution {
public:
    int pivotIndex(vector<int>& nums) {
        int n = nums.size();

        // Step 1: Compute total sum of entire array
        int sum = 0;
        for (int x : nums) sum += x;

        // Step 2: Loop with running left sum
        int left = 0; // prefix sum (starts at 0)

        for (int i = 0; i < n; i++) {
            // left += nums[i-1] done via: update left AFTER check at i=0
            // At i=0: left=0, right = sum - nums[0] - 0
            // At i=1: left += nums[0] first, then compute right

            // Update left to be prefix[i]
            if (i > 0) left += nums[i-1];

            // Compute right = sum - arr[i] - left
            int right = sum - nums[i] - left;

            // Check pivot condition
            if (left == right) return i;
        }
        return -1;
    }
};
```

Cleaner version — Update left at start of loop

```
// Same logic, slightly cleaner structure
class Solution {
public:
    int pivotIndex(vector<int>& nums) {
        int sum = 0;
        for (int x : nums) sum += x;
```

```

int left = 0;
for (int i = 0; i < nums.size(); i++) {
    // right = everything after i = total - left - nums[i]
    int right = sum - left - nums[i];

    if (left == right) return i;

    left += nums[i]; // update left for NEXT iteration
}
return -1;
};

// This version: left updated AFTER the check (for next i)
// At i=0: left=0, right=sum-0-nums[0]
// At i=1: left=nums[0], right=sum-nums[0]-nums[1] etc.

```

 **Full Dry Run** → arr = [1, 7, 3, 6, 5, 6], sum=28

i	left	right	Result
i=0	left=0	right=28-1-0=27	0≠27 → left+=1 → left=1
i=1	left=1	right=28-7-1=20	1≠20 → left+=7 → left=8
i=2	left=8	right=28-3-8=17	8≠17 → left+=3 → left=11
i=3	left=11	right=28-6-11=11	11==11  return 3



Final Answer: 3

Edge Cases to Remember

- **Single element:** arr = [5]. left=0, right=0. left==right → return 0
- **No pivot exists:** Loop ends without returning → return -1
- **All zeros:** Every index is a pivot → return 0 (leftmost)
- **Pivot at start (index 0):** left=0, right = sum - arr[0]. Works correctly
- **Pivot at end (index n-1):** left = sum - arr[n-1], right=0. Works correctly

10. The 4-Step Template (Works for ALL prefix problems)

Every prefix sum question — from simple pivot to complex hashmap — follows this exact 4-step structure. The only thing that changes is what data structure you use in Step 1.

Step	What to do
Step 1 Initialize Prefix Data Structure	Simple → variable (int left = 0) Medium → HashMap (unordered_map<int,int> freq) Hard → Deque or Segment Tree
Step 2 Loop over the Array	for (int i = 0; i < n; i++) Usually left to right (except when building suffix array)
Step 3 Update Prefix at each i	Variable: left += nums[i-1] or left += nums[i] HashMap: freq[prefixSum]++ Deque: pop front/back, push current
Step 4 Check Condition & Act	Compare left == right (pivot) Look up complement in hashmap Query deque for minimum/maximum window

Coming Up in This Series

Status	Problem	Data Structure	Difficulty
✓ Category 1 (Done!)	Pivot Index (LC 724)	Variable	Easy
🔍 Category 2 (Next)	Subarray Sum = k (LC 560) Subarray with Sum Divisible by k (LC 974)	HashMap	Medium
🔍 Category 3	Shortest Subarray with Sum ≥ k (LC 862) [with negatives]	Deque	Medium-Hard
🔍 Category 4	Count of Range Sum (LC 327)	Merge Sort on Prefix Array	Hard

🎯 The Big Picture

Prefix Sum sits in the **intermediate DSA zone** — harder than simple arrays, easier than DP/Graphs. But these intermediate patterns (Prefix Sum, Sliding Window, Kadane's, Two Pointer) are responsible for **~60% of interview questions**. Missing them = missing most of the interview.

Master these 4 categories and **400+ prefix sum problems become solvable**.

QUICK CHEAT SHEET:

```
        prefix[0] = 0
    prefix[i] = prefix[i-1] + arr[i-1]

        suffix[n-1] = 0
    suffix[i] = suffix[i+1] + arr[i+1]    ← built RIGHT to LEFT

    prefix[i] + arr[i] + suffix[i] = totalSum    (always!)
right = totalSum - left - arr[i]                (no suffix array needed)

    Negatives? → Sliding Window breaks → Use Prefix Sum
```